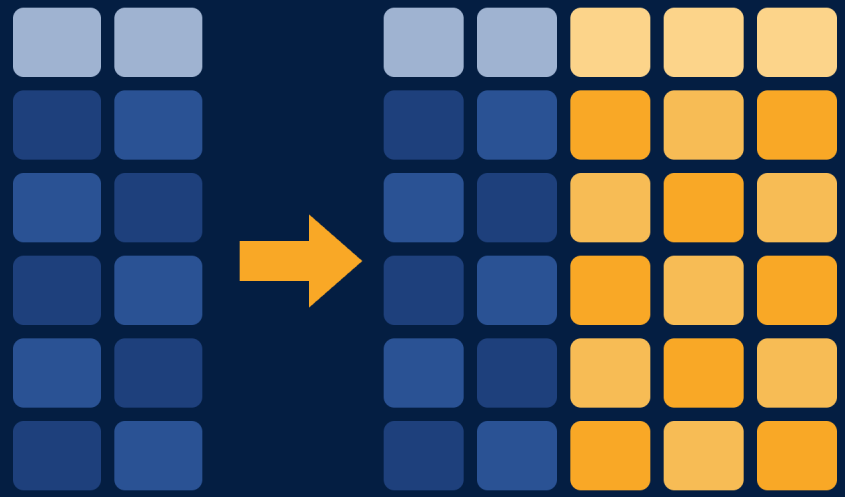
Please sit in the first 3 rows!

NITTANY DATA LABS

Feature Engineering

Turning raw columns into signal

BOOTCAMP #3



Attendance – “Feature Engineering”

[Attendance QR code]

THE BASICS

What is a feature?

A **feature** is a column a model learns from. **Feature engineering** is selecting, creating, and transforming those columns so patterns are easier to learn.



- Features are the model's **inputs**
- Better features, **better accuracy**

- **Create** new columns from old ones
- **Transform** columns into numbers

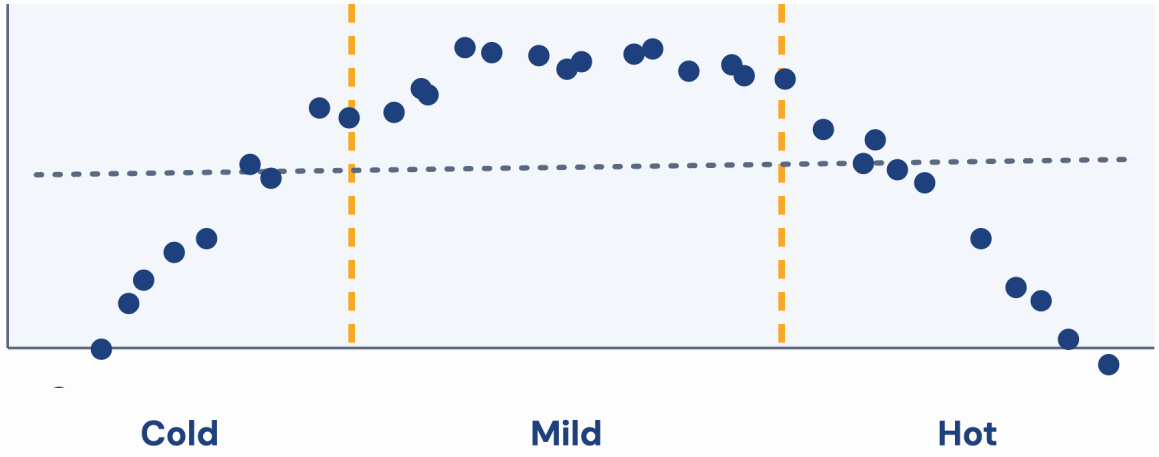
THE BASICS

Why feature engineering matters

- Models only see **what you give them**
- **Domain knowledge** spots what automation misses
- You **control** how data is represented
- Simple features are **easier to explain**

Same data, better feature

Shoppers vs. temperature · illustrative data · dotted line = best straight-line fit



A straight line misses the hump. Cold / mild / hot bins capture it.

THE ROADMAP

6 techniques for today

Start from clean, explored data, then add one technique at a time.

1. **Create features**
   Ratios, differences, flags

2. **One-hot encoding**
   Categories → 0/1 columns

3. **Label encoding**
   Ordered categories → numbers

4. **Binning**
   Numbers → ranges

5. **Feature splitting**
   One column → many

6. **Lag & sliding window**
   Past values for time series

STEP 1 OF 6

1 2 3 4 5 6

Create new features

- Combine columns using **domain knowledge**
- **Ratios:** price ÷ size
- **Differences:** sold date − listed date
- **Flags:** is_weekend, has_garage

Before → after: one new column

price	sqft	price_per_sqft (new)
350,000	1,750	200
420,000	1,400	300
280,000	2,000	140

```
df["price_per_sqft"] = df["price"] / df["sqft"]
df["is_big"] = (df["sqft"] > 1800).astype(int)
```

One-hot encoding

- One new **0/1 column** per category
- No fake order between categories
- Best for **unordered** columns: color, city
- Watch out: 100 cities = **100 columns**

color becomes 3 new columns, one per category

color	color_Blue	color_Green	color_Red
Red	0	0	1
Green	0	1	0
Blue	1	0	0

```
pd.get_dummies(df, columns=["color"], dtype=int)
```

STEP 3 OF 6

Label encoding

1 2 3 4 5 6

- Swap each category for **one number**
- Use it when the **order means something**
- One column instead of many
- No real order? Use **one-hot** instead

Good fit

Shirt size

`S → 0`

`M → 1`

`L → 2`

S < M < L is real

Risky

Color

`Red → 0`

`Green → 1`

`Blue → 2`

Model may think Blue > Red

```
df["size_code"] = df["size"].map({"S": 0, "M": 1, "L": 2})
```

STEP 4 OF 6

Binning

1 2 3 4 5 6

- Group nearby numbers into **ranges**
- Smooths out **noise** and extremes
- Captures curves, like the **shoppers** hump
- **pd.qcut** makes equal-sized bins

age → age_group

<18	18–34	35–59	60+
e.g. age 12	e.g. age 27	e.g. age 45	e.g. age 71

```
df["age_group"] = pd.cut(
    df["age"],
    bins=[0, 18, 35, 60, 120],
    labels=["<18", "18-34", "35-59", "60+"],
    right=False)
```

STEP 5 OF 6

Feature splitting

1 2 3 4 5 6

- Break one column into **useful parts**
- **Dates:** hour, weekday, month
- **Text:** first/last name, city/zip
- Models can't read a **timestamp string**

`2026-10-04 18:30` ⬇

`year = 2026` `month = 10` `weekday = Sun` `hour = 18`

`Smith, Jane` ⬇

`last = Smith` `first = Jane`

```
df["hour"] = df["date"].dt.hour
df["weekday"] = df["date"].dt.day_name()
df[["last", "first"]] = df["name"].str.split(", ", expand=True)
```

STEP 6 OF 6

1 2 3 4 5 6

Lag & sliding window features

- **Lag:** a past value as a new column
- **Window:** stats over the last N points
- Sort by time, and **only use the past**

Sliding window, size 3



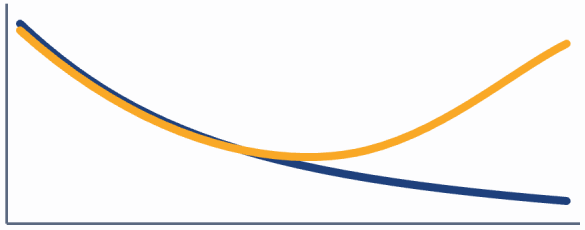
Navy = window (inputs) · gold = next value to predict

day	sales	sales_lag1	roll_mean_3
1	10	NaN	NaN
2	12	10	NaN
3	14	12	NaN
4	11	14	12.0
5	16	11	12.3

```
df = df.sort_values("date")
df["sales_lag1"] = df["sales"].shift(1)
df["roll_mean_3"] = df["sales"].shift(1).rolling(3).mean()
```

WATCH OUT

Three common traps



— training error — test error

Too many features

Enough features can memorize the training data and fail on new data. Add them one at a time and test each.

Mon | Tue | Wed | Thu? | ~~Fri~~

Predict Thursday with Mon–Wed only. Friday hasn't happened yet.

Peeking at the future

A feature built from future values or the target looks amazing in training, then fails for real.

Red = 0 | Green = 1 | Blue = 2

Is Blue “more” than Red?

Fake order

Label-encoding unordered categories invents a ranking the model may believe. Use one-hot instead.

BEST PRACTICES

Tools & habits

Use the right libraries

pandas to build columns, **scikit-learn** to transform and scale, **category_encoders** for categories.

Document every transform

Write down what you built and why, so anyone can reproduce and audit it.

Mix manual and automated

Design features with domain knowledge, then let automated feature selection prune them.

Measure each feature

Compare model scores and feature importance with and without it. Keep what helps.

YOUR TURN

Let’s engineer some features

Open the Colab notebook and work top to bottom. Stuck? Raise your hand or ask a neighbor.

1. Load the data and run df.head()
2. Create one ratio or difference feature
3. One-hot encode an unordered column
4. Label-encode an ordered column
5. Bin a numeric column with pd.cut
6. Split a date or text column
7. Add a lag and a rolling-mean feature
8. Bonus: train a model before and after

Interactive Notebook

[Colab QR code]

Thank you for coming!

Next meeting: [date] – Time Series Analysis & Visualization